narrowIN

CHNUG · 12 NOVEMBER 2026

From configuration tasks to declarative intent with ANSIBLE

Tomáš Horyl · narrowin

19:20–19:50



INTRODUCTION

Tomáš Horyl

Senior Network & Security Engineer · narrowin

- Two decades in network design and development
- Linux system deployment and maintenance
- Network services with a focus on security

narrowin.com/en/tomas

INTRODUCTION

IT/OT network & security for critical infrastructure

Energy, healthcare, utilities, research, manufacturing.
These sectors depend on reliable networks and protection of sensitive data.

EnBW
SWISSGAS
eniwa
Kernkraftwerk Gösgen
University of Basel
Universitätsspital Basel
Kantonsspital Baselland
Der Anschluss ans Leben iBB
ZWECKVERBAND BREITBANDVERSORGUNG LANDKREIS LÖRRACH
axians

THE FAMILIAR DESTINATION

One trunk
One access port

```
interface Ethernet1/1
 switchport trunk encapsulation dot1q
 switchport trunk native vlan 10
 switchport trunk allowed vlan 10,20,30
 switchport mode trunk
!
interface Ethernet1/2
 switchport mode access
 switchport access vlan 20
```

Describe the network you want.
Reuse the machinery that makes it real.

Desired-state excerpt, not a complete switch configuration.

ORDINARY ANSIBLE

Follow the variable into the task

Ethernet1/2 should be an access port in VLAN 20.

INPUT VARIABLES

```
access_ports:
  - name: Ethernet1/2
    vlan: 20
```

TASK EXCERPT

```
- name: Configure access ports
  cisco.ios.ios_l2_interfaces:
    config:
      - name: "{{ item.name }}"
        mode: access
        access:
          vlan: "{{ item.vlan | int }}"
    state: merged
  loop: "{{ access_ports }}"
```

Readable so far. Reviewing the request means following inputs into task parameters.

ORDINARY ANSIBLE

Now follow the trunk settings

The destination has not changed. The operator follows another input and payload.

INPUT VARIABLES

```yaml
trunk_ports:
  - name: Ethernet1/1
    encapsulation: dot1q
    native_vlan: 10
    allowed_vlans: "10,20,30"
```

TASK EXCERPT

```yaml
- name: Configure trunk ports
  cisco.ios.ios_l2_interfaces:
    config:
      - name: "{{ item.name }}"
        mode: trunk
        trunk:
          encapsulation: "{{ item.encapsulation }}"
          native_vlan: "{{ item.native_vlan | int }}"
          allowed_vlans: "{{ item.allowed_vlans }}"
    state: merged
  loop: "{{ trunk_ports }}"
```

What happens when shared defaults, profiles and host exceptions supply these inputs?

COMPOSITION PRESSURE

Same ports
More places to look

Trace only Ethernet1/1: defaults, a selected profile and the host input.

SHARED DEFAULTS

```
switchport_defaults:
  mode: access
  access_vlan: 10
  trunk_encapsulation: dot1q
  native_vlan: 10
```

PROFILES AND HOST INPUTS

```
switchport_profiles:
  edge: {}
  uplink:
    mode: trunk
    allowed_vlans: [10, 20, 30]

switchport_overrides:
  Ethernet1/1:
    profile: uplink
  Ethernet1/2:
    access_vlan: 20
```

The host selects a profile. The final trunk values still need to be established.

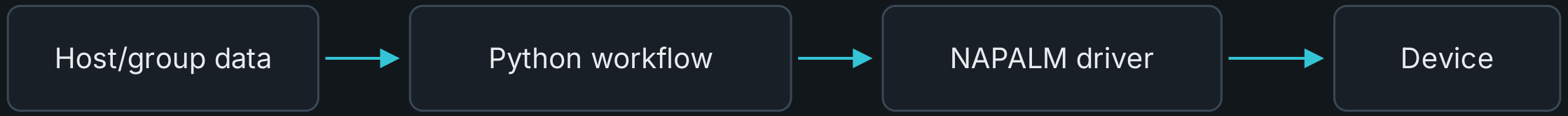
A PROGRAM INSIDE INVENTORY

One template
Both ports

```
ios_switchport_config: >-
  {%- set result = namespace(ports=[]) -%}
  {%- for name, override in switchport_overrides.items() -%}
    {%- set profile_name = override.get('profile', 'edge') -%}
    {%- set settings = switchport_defaults
        | combine(switchport_profiles[profile_name]) | combine(override) -%}
    {%- set port = {'name': name, 'mode': settings.mode} -%}
    {%- if settings.mode == 'access' -%}
      {%- set port = port | combine({'access': {'vlan': settings.access_vlan}}) -%}
    {%- elif settings.mode == 'trunk' -%}
      {%- set port = port | combine({'trunk': {
        'encapsulation': settings.trunk_encapsulation,
        'native_vlan': settings.native_vlan,
        'allowed_vlans': settings.allowed_vlans | map('string') | join(',')
      }}) -%}
    {%- endif -%}
    {%- set result.ports = result.ports + [port] -%}
  {%- endfor -%}
  {{ result.ports }}
```

FIRST ALTERNATIVE: A PYTHON-ORIENTED WORKFLOW

Nornir for inventory
NAPALM for device integration



INVENTORY AND EXECUTION

Nornir

Hosts, groups and defaults.
Run a Python task per host.
Collect per-host results.

RENDERING AND POLICY

Application code

Compose the configuration.
Render native CLI.
Coordinate driver calls and checks.

DEVICE INTEGRATION

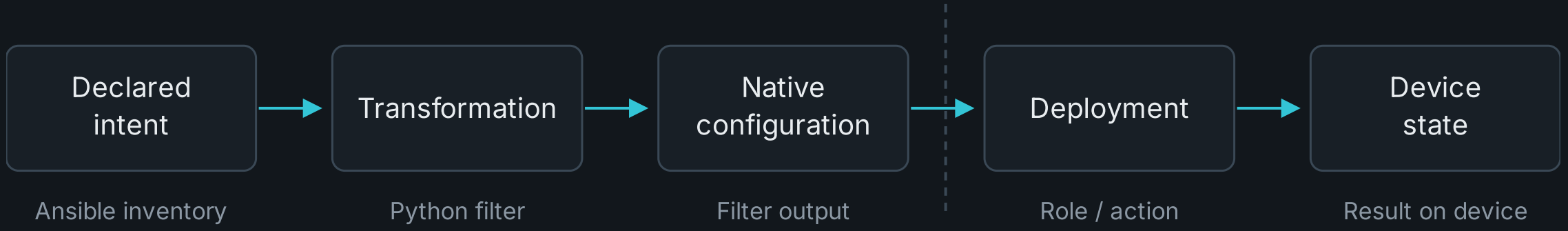
NAPALM

Render and stage templates.
Common methods to load, compare and commit configuration.
Getters for operational data.

The application owns the workflow. Driver capabilities vary.

CHOSEN DIRECTION · REUSABLE ARCHITECTURE

One framework.
Our Ansible implementation.



Shared responsibilities. Platform-specific implementations.

INTENT TO DEVICE STATE · STEP 1 OF 5

Declared intent

Decide what should exist—and what may change.



DESIRED COMMANDS IN INVENTORY

Inherit. Override. Remove explicitly.

GROUP_VARS/SWITCHES.YML

```
ios_group:
  update:
    interface:
      Ethernet1/1:
        switchport mode: access
        switchport access vlan: 10
      Ethernet1/2:
        switchport mode: access
        switchport access vlan: 10
```

HOST_VARS/SWITCH1.YML

```
ios_host:
  delete:
    interface:
      Ethernet1/1:
        switchport access vlan: {}
  update:
    interface:
      Ethernet1/1:
        switchport trunk encapsulation: dot1q
        switchport mode: trunk
        switchport trunk native vlan: 10
        switchport trunk allowed vlan: "10,20,30"
      Ethernet1/2:
        switchport access vlan: 20
```

Group first, host last.
Caller role: `narrowin.cisco_ios_intent.intent`. The implementation work has not disappeared.

INTENT TO DEVICE STATE · STEP 2 OF 5

Transformation

Turn supplied data into native configuration.



WHAT THE DECLARATION CONTROLS

If I leave a setting out, will it survive?

A manual description exists. Inventory does not mention it. What determines the result?

CISCO IOS · COMPLETE FILE

Source decides

Baseline: restore the old label.
Running: retain the label
in the captured snapshot.
Both use complete-file
replacement.

NOKIA SR LINUX · PATHS

Target decides

An unrelated path stays
outside the requested change.
Replace/delete its parent path:
the description is then
inside the owned scope.

CUMULUS NVUE · POC

NVUE Root decides

The declared NVUE root
is authoritative.
Omitted configuration can
be removed or return
to its NVUE default.

Review what may change—and what must stay.

SOURCE FOR UNDECLARED SETTINGS

Restore the old label—or keep the operator's manual change?

Both enforce switchport intent. Choose what supplies the unowned description.

REVIEWED BASELINE

```
interface Ethernet1/2
 description BASELINE-LABEL
```

source: baseline

Candidate restores BASELINE-LABEL.

Use when:

The reviewed baseline is authoritative.

Out-of-band changes must be reverted.

CURRENT RUNNING SNAPSHOT

```
interface Ethernet1/2
 description MANUAL-LABEL
```

source: running

Candidate retains MANUAL-LABEL.

Use when:

Adopting a brownfield device gradually.

Keeping reviewed manual changes.

Unowned by inventory does not mean protected from full-file replacement.

WHY SEPARATE TRANSFORMATION?

Check the rules without touching a switch

Supply known input. Compare the result with an expected artifact.

Implementation	Given this input…	Expect this output…
Cisco IOS	Shared access VLAN 10; host explicitly deletes that command on Ethernet1/1.	No access-VLAN command on Ethernet1/1 in the candidate.
Nokia SR Linux	Delete the description of `ethernet-1/51` in an already-resolved tree.	A delete operation for `/interface[name=ethernet-1/51]/description`
Cumulus NVUE PoC	An inherited pre-login banner; a later layer sets it to `null`.	The banner key is absent from the final NVUE root.

Separate transformation. Test it without deployment.

OFFLINE RENDERER TEST

Test the renderer without a switch

GIVEN

Known inputs

Invented source configuration.
Known inventory settings.

WHEN

Run the renderer

Run the Python function.
No SSH. No upload.

THEN

Check the output

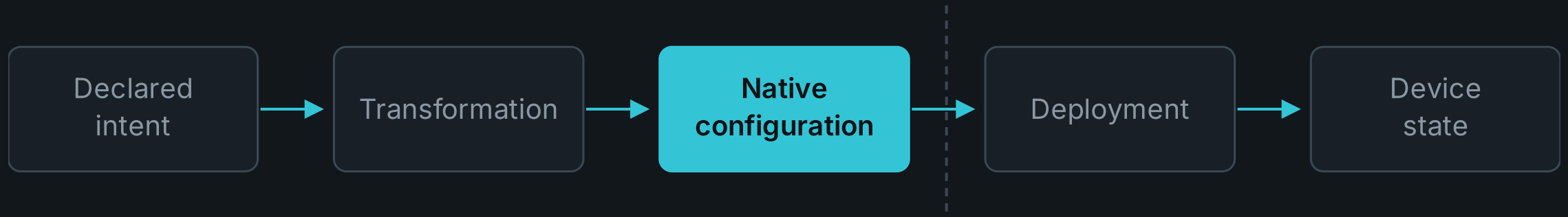
Does the result match
the expected configuration?

Catch rendering mistakes before deployment.

INTENT TO DEVICE STATE · STEP 3 OF 5

Native configuration

Review the native input and its proposed changes.



NEXT VALIDATION STEP · IOS DIFF

Ask the device what would change

IOS compares its running configuration with the supplied candidate.

```
!Contextual Config Diffs:
interface Ethernet1/2
 +switchport access vlan 30
interface Ethernet1/2
 -switchport access vlan 20
```

This is what IOS returned

- Existing access VLAN: 20.
+ Candidate access VLAN: 30.

One change, not two steps

Proposed: access VLAN 20 → 30 on Ethernet1/2.
This comparison does not apply the change.

Fixtures check the renderer. IOS reports the proposed difference.

INTENT TO DEVICE STATE · STEP 4 OF 5

Deployment

Use the native path. Check the outcome.



REUSE DEPLOYMENT ACROSS FEATURES

Describe the target
Reuse the native deployment path

Change the native configuration data—not the deployment machinery for every feature.

CISCO IOS

IOS reconciles running state

Caller stages a complete file. IOS determines and executes the required CLI changes.

NOKIA SR LINUX

SR Linux processes native operations

Caller uses vendor modules. SR Linux processes the targeted native operations.

CUMULUS NVUE · POC

NVUE applies the revision

Caller stages the root tree in a revision. NVUE applies its managed configuration.

New supported settings reuse the same deployment path.

Reuse stays within each platform's native contract—not one universal API.
Nokia/NVUE: source-assessed paths, not new live qualification.

ERROR CHECKS · THESE IMPLEMENTATIONS

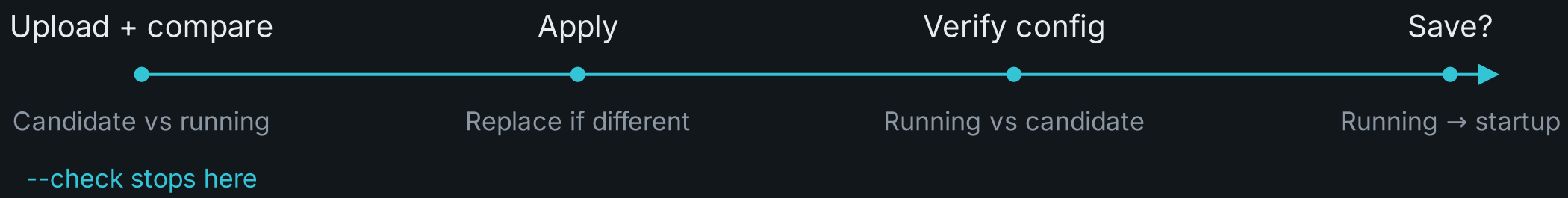
What gets checked—and when?

Implementation	Before apply	Apply outcome
Cisco IOS	Structure/conflicts + IOS diff. No general argument check.	Fresh IOS diff checks running against the candidate.
Nokia SR Linux	Version guard. Device validates update/replace. Device diff covers all operations.	Device transaction result. No separate convergence check.
Cumulus NVUE PoC	Partial NVUE path checks. Revision staging + diff.	Wait for NVUE to report apply/save completion. No separate convergence check.

No check here proves that traffic works.

REUSE DEPLOYMENT · CISCO IOS

Apply to running.
Save to startup.



Optional rollback timer · starts at Apply

confirm: true	Auto-cancel after config checks; then save if enabled.
confirm: false	Caller confirms, or expiry attempts to restore previous running. Requires save: false.

Defaults: no timer · save enabled. `--diff` alone can apply.

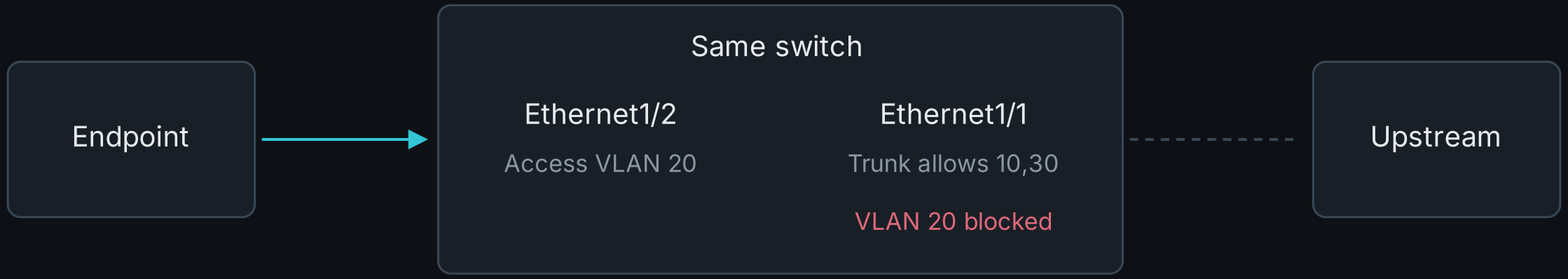
Device state

Verify the service, not just the configuration.



A VALID CONFIGURATION CAN EXPRESS THE WRONG DESIGN

Automation can correctly apply the wrong intention.




```
switchport trunk allowed vlan 10,30
```

ADOPTION

When is this approach useful?

Useful when…	You still need to…
Shared settings and device exceptions are hard to follow	Define how settings combine—and what may change
Operators need to see what will change	Write and review platform-specific configuration
You want to reuse deployment across features	Maintain the code and test supported software versions

Reuse the machinery. Keep the rules clear.

THREE TAKEAWAYS

Describe the network.
Reuse the machinery.

01 **Read the desired configuration in inventory**
Keep declarations separate from implementation.

02 **Review what will change before applying**
Check the native input, device diff and what may change.

03 **Verify the service**
Not just that configuration was applied.



Thank you

Questions and discussion

Tomáš Horyl

Senior Network & Security Engineer · narrowin

E-MAIL info@narrowin.com

PROFILE narrowin.com/en/tomas

WEB narrowin.com

BACKUP · CVD-FABRIC-INTENT.YML · SELECTED FIELDS

One playbook.
Ordered fabric contributions.

PLAY AND ROLE

```
- name: Reconcile the isolated Cisco campus fabric
  hosts: ios_cvd_fabric
  gather_facts: false
  serial: 1
  any_errors_fatal: true
  roles:
    - narrowin.cisco_ios_intent.intent
```

VARIABLES IN THE SAME PLAY

```
  vars:
    ansible_command_timeout: 120
    cisco_ios_intent_group_order:
      - ios_cvd_fabric
      - ios_cvd_vteps
      - ios_cvd_core
      - ios_cvd_spines
      - ios_cvd_leaves
      - ios_cvd_transports
```

BACKUP · IOS_CVD_FABRIC.YML · SELECTED FIELDS

Shared fabric settings

COMMON SERVICES AND INTERFACES

```
ios_cvd_fabric:
  update:
    ip multicast-routing:
    ip cef:
    interface:
      Loopback0:
        description: CVD-LOOPBACK-0
        ip pim sparse-mode:
        ip ospf 1 area: '0'
        shutdown: false
      Ethernet0/1:
        no ip address: false
        ip pim sparse-mode:
        ip ospf network: point-to-point
        ip ospf dead-interval: '4'
        ip ospf hello-interval: '1'
        ip ospf 1 area: '0'
        shutdown: false
```

ROUTING · CONTINUES UNDER UPDATE

```
    router ospf:
      '1':
        passive-interface: default
        no passive-interface Ethernet0/1:
        no passive-interface Ethernet0/2:
    ip pim rp-address: 172.16.101.100
    ip pim ssm: default
```

BACKUP · IOS_CVD_LEAVES.YML · SELECTED FIELDS

Shared leaf settings

SERVICE ATTACHMENT AND UPLINK COSTS

```
ios_cvd_leaves:
  update:
    bridge-domain:
      '101':
        member Ethernet0/3: service-instance 101
    interface:
      Ethernet0/1:
        description: CVD-TO-TRANSPORT1
        ip ospf cost: '10'
      Ethernet0/2:
        description: CVD-TO-TRANSPORT2
        ip ospf cost: '100'
      Ethernet0/3:
        service instance:
          101 ethernet:
            encapsulation: untagged
        description: CVD-HOST-101
        no ip address:
```

EVPN PEERS · CONTINUES UNDER UPDATE

```
    router bgp:
      '6510':
        address-family:
          l2vpn evpn:
            neighbor 172.16.101.3 activate:
            neighbor 172.16.101.3 send-community: both
            neighbor 172.16.101.3 encap: vxlan
            neighbor 172.16.101.4 activate:
            neighbor 172.16.101.4 send-community: both
            neighbor 172.16.101.4 encap: vxlan
        neighbor 172.16.101.3 remote-as: '6510'
        neighbor 172.16.101.3 update-source: Loopback0
        neighbor 172.16.101.4 remote-as: '6510'
        neighbor 172.16.101.4 update-source: Loopback0
```

BACKUP · CLAB-IOS-CVD-FABRIC-LEAF1.YML · SELECTED FIELDS

leaf1: identity and exceptions

ADDRESSES, ROUTE DISTINGUISHER AND OVERRIDE

```
ios_host:
  update:
    vrf definition:
      CORP:
        rd: 172.16.101.7:101
    interface:
      Loopback0:
        ip address: 172.16.101.7 255.255.255.255
      Loopback1:
        ip address: 172.16.102.7 255.255.255.255
      Ethernet0/1:
        description: CVD-TO-TRANSPORT1-MAINTENANCE
        ip address: 10.101.10.15 255.255.255.254
        ip ospf cost: '200'
      Ethernet0/2:
        ip address: 10.101.10.19 255.255.255.254
      BDI101:
        ip address: 10.101.101.7 255.255.255.0
```

ROUTER IDENTITY AND PEER · UNDER UPDATE

```
    router ospf:
      '1':
        router-id: 172.16.101.7
    router bgp:
      '6510':
        address-family:
          l2vpn evpn:
            neighbor 172.16.101.9 activate:
            neighbor 172.16.101.9 send-community: both
            neighbor 172.16.101.9 encap: vxlan
        bgp router-id: 172.16.101.7
        neighbor 172.16.101.9 remote-as: '6510'
        neighbor 172.16.101.9 update-source: Loopback0
```

BACKUP · VALIDATION ASSURANCES

Three implementations.
Not the same validation guarantee.

	Cisco IOS	Nokia SR Linux	Cumulus NVUE PoC
Local checks	Structure, ownership and supported CLI rendering	Structure and native paths using list-key metadata	Recognizable paths in a selected OpenAPI catalog
Before apply	Checksum and native diff; no general semantic validation	Live update/replace validation, then all-operation diff preflight	Stage a revision and obtain a diff; path checks are incomplete
Blind spot	Invalid arguments may reach replacement	Catalog freshness does not prove full compatibility or correct design	Unknown final leaves can pass; types and dependencies are not fully checked

A validation result must say what it proves.
None of these checks alone proves a working network.